IoT System Map

Room Temperature & Humidity Monitoring System

SWYNEX Internship — Task 1

A conceptual IoT architecture documenting the sensor, controller, connectivity, cloud processing, dashboard, and end-to-end data flow.

Design proposal only. No physical hardware or live measurements are connected.

1. Objective

Design a simple system that monitors room temperature and humidity, sends readings to a cloud service, and displays current and historical data on a dashboard.

2. System Architecture

Layer	Component	Responsibility
Sensing	DHT22 sensor	Measures temperature and relative humidity.
Edge/controller	ESP32	Reads sensor values and prepares a timestamped payload.
Connectivity	Wi-Fi	Connects the device to the network.
Messaging	MQTT or HTTP	Transfers data to a cloud endpoint.
Cloud/backend	API + database / IoT platform	Validates and stores readings.
Application	Web dashboard	Shows latest values, charts, and alerts.

3. End-to-End Data Flow

1. The DHT22 measures room temperature and humidity.
2. The ESP32 reads the sensor output at a configured interval, such as every 30 seconds.
3. The ESP32 packages the values as JSON and sends them over Wi-Fi using MQTT or HTTP.
4. A cloud endpoint validates the incoming message and stores it with a timestamp.
5. The dashboard retrieves recent and historical measurements for display.
6. If temperature exceeds a configured threshold, the dashboard can highlight the value or trigger an alert.

4. Example Data Payload

```
{
"device_id": "room-01",
"temperature_c": 27.4,
"humidity_percent": 58,
"timestamp": "2026-10-27T06:30:00Z"
}
```

The values above are illustrative only; they are not real sensor readings.

5. Use Case

A room owner can check environmental conditions remotely. Historical charts may reveal recurring heat patterns, and threshold alerts can prompt ventilation or cooling.

6. Security and Reliability

• Use authenticated device credentials and encrypted transport (TLS/HTTPS or MQTT over TLS).

• Never publish passwords, API keys, or device secrets in the repository.

• Validate incoming readings and reject malformed messages.

• Retry transmission after temporary network failure; buffer readings if the device supports it.

• Use consistent timestamps and monitor device connectivity.

7. Assumptions and Limitations

This is a design document, not a deployed prototype. No physical sensor, ESP32, cloud database, or live dashboard is connected. MQTT/HTTP and the cloud platform are proposed options; the final implementation depends on the service selected.

8. Learning Outcomes

The design identifies the sensing, edge, connectivity, cloud, and application layers; traces data from sensor to dashboard; and considers basic security, validation, and reliability.